



FACULDADE DE TECNOLOGIA SENAI GASPAR RICARDO JÚNIOR

INTEGRAÇÃO DE PROTOCOLO OPC-UA COM INTERFACE WEB EM AMBIENTE INDUSTRIAL CONTAINERIZADO

INTEGRATION OF OPC-UA PROTOCOL WITH WEB INTERFACE IN A CONTAINERIZED INDUSTRIAL ENVIRONMENT

Paola Oliveira ¹

²

RESUMO

Este artigo apresenta o desenvolvimento de um sistema de interface industrial baseado no protocolo OPC-UA, integrado a uma API RESTful em Node.js e a uma interface web em Next.js, com toda a infraestrutura orquestrada via Docker Compose. O sistema realiza a leitura em tempo real de variáveis industriais — temperatura, pressão, umidade, sensor de presença e trava de segurança — por meio de um servidor OPC-UA implementado em Python. O Node-RED atua como middleware de integração, coletando os dados do servidor OPC-UA e os encaminhando à API REST, que os armazena em memória e os disponibiliza ao frontend. A interface web apresenta um dashboard de monitoramento e um módulo de gerenciamento de dispositivos. Os resultados demonstram a viabilidade da solução para aplicações de automação industrial, com arquitetura modular, escalável e de fácil implantação.

Palavras-chave: OPC-UA; IoT Industrial; API REST; Node-RED; Docker; Next.js; Automação.

ABSTRACT

This paper presents the development of an industrial interface system based on the OPC-UA protocol, integrated with a RESTful API in Node.js and a web interface in

Next.js, with the entire infrastructure orchestrated via Docker Compose. The system performs real-time reading of industrial variables — temperature, pressure, humidity, presence sensor, and safety lock — through an OPC-UA server implemented in Python. Node-RED acts as an integration middleware, collecting data from the OPC-UA server and forwarding it to the REST API, which stores it in memory and makes it available to the frontend. The web interface features a monitoring dashboard and a device management module. Results demonstrate the feasibility of the solution for industrial automation applications, with a modular, scalable, and easy-to-deploy architecture.

Keywords: OPC-UA; Industrial IoT; REST API; Node-RED; Docker; Next.js; Automation.

1 Introdução

A automação industrial contemporânea exige sistemas capazes de coletar, processar e exibir dados de equipamentos em tempo real, integrando camadas de campo, controle e supervisão. Nesse contexto, o protocolo OPC-UA (OPC Unified Architecture) emerge como padrão consolidado para comunicação entre dispositivos industriais, oferecendo interoperabilidade, segurança e independência de plataforma (OPC Foundation, 2017).

Paralelamente, a crescente adoção de tecnologias web no ambiente industrial — impulsionada pela Indústria 4.0 — abre espaço para soluções de monitoramento acessíveis via navegador, sem a necessidade de softwares proprietários. A combinação de protocolos industriais com arquiteturas modernas de software, como APIs RESTful e frameworks JavaScript, representa uma tendência relevante para o desenvolvimento de sistemas de interface homem-máquina (IHM) de nova geração.

Este trabalho apresenta a concepção e implementação de um sistema integrado que utiliza OPC-UA como protocolo de aquisição de dados, Node-RED como middleware de integração, uma API RESTful construída com Node.js/Express como camada de negócio, e Next.js como interface web de supervisão. Toda a infraestrutura é containerizada com Docker, garantindo portabilidade e facilidade de implantação.

O objetivo é demonstrar a viabilidade técnica dessa arquitetura para aplicações

industriais reais, evidenciando a integração entre as camadas e os resultados obtidos no monitoramento de variáveis simuladas.

2 Revisão de Literatura

O protocolo OPC-UA foi concebido pela OPC Foundation como evolução do OPC clássico, superando limitações de plataforma e adicionando suporte a segurança por certificados, modelagem semântica de dados e transporte via TCP ou HTTPS (Mahnke et al., 2009). Segundo Cavalieri e Carro (2015), o OPC-UA tornou-se o protocolo de referência para comunicação M2M (machine-to-machine) em ambientes industriais, sendo adotado em iniciativas como a Plataforma Industrie 4.0.

No contexto da Internet das Coisas Industrial (IIoT), a integração entre protocolos de campo e camadas de aplicação web é viabilizada por ferramentas de middleware como o Node-RED. Conforme descrito por Blackstock e Lea (2014), o Node-RED oferece uma abordagem de programação visual baseada em fluxos, adequada para prototipagem rápida de pipelines de dados IoT.

A utilização de APIs RESTful como camada de abstração entre hardware e interface de usuário é amplamente recomendada na literatura. Aguiar (2023) destaca que APIs REST permitem que dispositivos físicos exponham seus dados de forma padronizada, facilitando a integração com aplicações web e móveis. Rodrigues (2024) complementa, afirmando que a adoção de REST em sistemas IoT industriais garante interoperabilidade entre diferentes plataformas e tecnologias.

A containerização com Docker, por sua vez, representa uma estratégia consolidada para isolamento e portabilidade de serviços. Segundo Merkel (2014), containers permitem que aplicações sejam empacotadas com todas as suas dependências, eliminando problemas de compatibilidade entre ambientes de desenvolvimento e produção — aspecto crítico em sistemas industriais que precisam ser implantados em diferentes infraestruturas.

3 Metodologia

O sistema foi desenvolvido com arquitetura em quatro camadas, orquestradas por Docker Compose, conforme descrito a seguir.

3.1 Servidor OPC-UA (Python)

O servidor OPC-UA foi implementado em Python utilizando a biblioteca `opcua` (versão 0.98.13). O servidor expõe um namespace próprio (`http://ads.freeopcua.server`) com um objeto `Sensor` contendo cinco variáveis: `temperatura` (`Double`), `pressao` (`Double`), `umidade` (`Double`), `sensor_presenca` (`Boolean`) e `trava_seguranca` (`Boolean`).

Os valores são atualizados a cada segundo com dados simulados dentro de faixas representativas de um ambiente industrial: temperatura entre 18°C e 30°C, pressão entre 0,8 e 1,5 bar, umidade entre 35% e 60%. Os valores booleanos são alternados aleatoriamente, simulando eventos de presença e acionamento de trava. O servidor escuta na porta TCP 4840 em todas as interfaces (0.0.0.0), sem política de segurança, aceitando conexões anônimas.

3.2 Middleware Node-RED

O Node-RED atua como bridge entre o servidor OPC-UA e a API REST. O fluxo implementado realiza leituras periódicas (a cada 2 segundos) das cinco variáveis via nós `OpcUa-Client` e `OpcUa-Item`, armazenando os valores lidos no contexto de fluxo (`flow.set`). Um nó `inject` separado, também com intervalo de 2 segundos, monta o payload JSON e realiza um HTTP POST para o endpoint `/newData` da API.

A separação entre leitura e envio foi implementada para garantir que os dados coletados das diferentes variáveis OPC-UA sejam consolidados antes do envio, evitando requisições com dados parciais.

3.3 API RESTful (Node.js/Express)

A API foi construída com Express.js e expõe os seguintes endpoints:

- **POST /newData** — recebe e armazena leituras de sensores enviadas pelo Node-RED
- **GET /iot** — retorna o histórico completo de leituras
- **GET /sensor/:id** — retorna uma leitura específica por ID
- **PUT /sensor/:id** — atualiza uma leitura existente

- **GET /devices** — lista os dispositivos cadastrados
- **POST /devices** — cadastra um novo dispositivo
- **PUT /devices/:id** — atualiza dados de um dispositivo
- **DELETE /devices/:id** — remove um dispositivo
- **PATCH /devices/:id/status** — atualiza o status operacional (online, offline, alerta)

Os dados são mantidos em memória (arrays JavaScript), sem persistência em banco de dados, adequado para o escopo de prototipagem do projeto. A API escuta na porta 8080 e possui CORS habilitado para permitir requisições do frontend.

3.4 Interface Web (Next.js)

O frontend foi desenvolvido com Next.js 16 e React 19, utilizando Tailwind CSS para estilização. A interface é composta por três módulos:

- **Dashboard de Sensores** — exibe a última leitura em cards individuais por variável e um histórico tabular com todas as leituras, atualizado automaticamente a cada 2 segundos via `setInterval`
- **Gerenciar Dispositivos** — permite cadastrar, listar, atualizar e remover dispositivos industriais, com controle de status (online/offline/alerta)
- **Gerenciar Usuários** — módulo para controle de acesso ao sistema

O design adota paleta escura no estilo terminal industrial, com indicadores visuais coloridos para estados críticos (vermelho para trava acionada, verde para presença detectada).

3.5 Infraestrutura Docker

Os quatro serviços são orquestrados via Docker Compose com dependências explícitas:

Tabela 1: Serviços Docker e suas configurações

Serviço	Porta	Dependência
opcua-server	4840	—
api-ifaci	8080	opcua-server
frontend-ifaci	3000	api
nodered-ifaci	1880	api

A comunicação interna entre containers é feita pelos nomes dos serviços definidos no Compose, eliminando a necessidade de endereços IP fixos.

4 Resultados e Discussões

O sistema foi implantado com sucesso via `docker compose up --build`, com todos os quatro serviços inicializando na ordem correta definida pelas dependências.

4.1 Aquisição de Dados OPC-UA

O servidor OPC-UA iniciou corretamente na porta 4840, publicando leituras a cada segundo. Os logs confirmaram a geração contínua de valores dentro das faixas esperadas:

Tabela 2: Exemplo de leituras geradas pelo servidor OPC-UA

Variável	Valor Exemplo	Unidade
Temperatura	23,09	°C
Pressão	1,27	bar
Umidade	49,39	%
Sensor Presença	True	—
Trava Segurança	True	—

4.2 Integração Node-RED e API

O Node-RED conectou-se ao servidor OPC-UA usando o hostname `opcua-server:4840` (nome do serviço Docker). As leituras foram coletadas e enviadas com sucesso ao

endpoint `POST /newData` da API. O histórico de dados ficou disponível em `GET /iot` para consumo pelo frontend.

Um ponto relevante identificado durante os testes foi o problema de timing no início do fluxo: o nó de envio à API disparava antes que os nós de leitura OPC-UA completassem o primeiro ciclo, gerando avisos de dados ausentes. Esse comportamento é esperado no primeiro ciclo e se normaliza a partir do segundo intervalo (2 segundos).

4.3 Interface Web

O dashboard exibiu corretamente as leituras em tempo real, com atualização automática a cada 2 segundos. Os cards de última leitura apresentaram os valores numéricos com duas casas decimais, e os indicadores booleanos exibiram estados visuais distintos com codificação por cores. A tabela de histórico mostrou todas as leituras em ordem cronológica inversa, com destaque visual na linha mais recente.

O módulo de gerenciamento de dispositivos permitiu cadastrar, editar, remover e alternar o status de dispositivos, com todas as operações refletidas imediatamente na lista.

4.4 Discussão

A arquitetura adotada demonstrou ser adequada para o escopo do projeto, com clara separação de responsabilidades entre as camadas. A containerização facilitou a implantação e eliminou problemas de compatibilidade de dependências. O uso do Node-RED como middleware mostrou-se eficiente para integração rápida entre OPC-UA e REST, sem necessidade de código customizado para o protocolo de comunicação industrial.

Como limitação, a ausência de persistência em banco de dados implica perda de histórico ao reiniciar os containers. Para um ambiente de produção real, seria necessário integrar um banco de dados como PostgreSQL ou InfluxDB para séries temporais. Adicionalmente, a política de segurança do servidor OPC-UA foi desabilitada para simplificar a prototipagem — em produção, o uso de certificados X.509 seria obrigatório.

5 Conclusão

O projeto demonstrou a viabilidade de integrar o protocolo industrial OPC-UA a uma plataforma web moderna utilizando ferramentas de código aberto e containerização. A arquitetura em quatro camadas — servidor OPC-UA, middleware Node-RED, API RESTful e interface Next.js — mostrou-se coesa e funcional, com comunicação entre serviços operando corretamente dentro da rede Docker.

A solução desenvolvida tem potencial de aplicação em cenários reais de supervisão industrial de pequeno e médio porte, especialmente em contextos onde a implantação rápida e o acesso via navegador são requisitos prioritários. Como trabalhos futuros, pretende-se adicionar persistência em banco de dados de séries temporais, implementar autenticação na API e no servidor OPC-UA, e desenvolver visualizações gráficas de histórico com bibliotecas como Recharts ou Chart.js.

Agradecimentos

Agradeço à instituição SENAI e ao orientador Gabriel pelo suporte e direcionamento durante o desenvolvimento deste projeto.

AGUIAR, Claudio. API REST e Internet das Coisas: Conectando o mundo físico ao digital. DIO, 2023.

BLACKSTOCK, Michael; LEA, Rodger. IoT Interoperability: A Hub-based Approach. In: *Proceedings of the International Conference on the Internet of Things (IoT)*. 2014.

CAVALIERI, Salvatore; CARRO, Gianfranco. Revisiting OPC UA Client-Server and Publish-Subscribe Communication Models for Industry 4.0. *IEEE Transactions on Industrial Informatics*, 2020.

MAHNKE, Wolfgang; LEITNER, Stefan-Helmut; DAMM, Matthias. *OPC Unified Architecture*. Springer, 2009.

MERKEL, Dirk. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, v. 2014, n. 239, 2014.

OPC FOUNDATION. *OPC Unified Architecture Specification*. OPC Foundation, 2017.

RODRIGUES, Thaís dos Santos. Expansão de funcionalidades em sistema IoT de controle de acesso através de API HTTP. Universidade Federal de Pernambuco, 2024.